

FoldAudit: Batch Data Integrity Verification via Polynomial Folding with Tight Soundness

Abstract—

Index Terms—Polynomial Commitment, Vector Commitment, Data Management

I. INTRODUCTION

The exponential growth of global data generation has driven a paradigm shift toward decentralized and cloud-based storage architectures. By outsourcing data to remote storage providers, individuals and organizations significantly reduce local hardware maintenance costs while benefiting from elastic scalability, geographic redundancy, and high availability. However, this outsourcing model inherently relinquishes physical control over the stored assets, exposing them to multifaceted risks such as hardware degradation, silent data corruption, and economically motivated data deletion or compression by untrusted storage providers. To mitigate these threats and restore trust in distributed storage ecosystems, Provable Data Possession (PDP) and Proof of Retrievability (PoR) schemes have emerged as foundational cryptographic primitives. These protocols enable data owners or delegated auditors to efficiently verify the integrity of outsourced data without retrieving the entire dataset, thereby ensuring continuous data availability, regulatory compliance, and accountability in modern storage networks.

Recent advancements in cryptographic primitives have significantly optimized the efficiency and scalability of data integrity auditing. Among these, Polynomial Commitment Schemes (PCS), particularly the Kate-Zaverucha-Goldberg (KZG) construction, have been widely adopted in state-of-the-art PDP designs. By encoding data blocks as coefficients of low-degree polynomials over a finite field, PCS enables compact, constant-size commitments and supports homomorphic aggregation of multiple block tags. This algebraic structure allows cloud servers to generate succinct proofs of possession through polynomial evaluation and quotient computation, while verifiers can validate correctness using a minimal number of bilinear pairings. Consequently, polynomial commitments have become the cornerstone of lightweight, publicly verifiable auditing frameworks, drastically reducing communication bandwidth and on-chain storage overhead compared to traditional hash-based or signature-based approaches. Their compatibility with structured reference strings (SRS) and efficient synthetic division algorithms further positions them as ideal candidates for large-scale storage verification.

Despite these cryptographic improvements, practical storage deployments frequently involve massive-scale data repositories comprising thousands of files distributed across multiple storage nodes, providers, or geographic regions. Conventional

PDP protocols process each file independently, requiring separate proof generation, transmission, and verification rounds for every audited dataset. This linear scaling introduces severe computational and economic bottlenecks, particularly in blockchain-assisted auditing systems where smart contract execution costs (gas) and on-chain proof storage scale directly with the number of files. As audit frequencies increase and batch sizes grow, the cumulative overhead of independent verifications rapidly degrades system throughput, inflates operational expenses, and strains verifier resources. Therefore, developing a secure and efficient multi-file batch verification mechanism that compresses multiple integrity checks into a single, constant-size cryptographic operation has become a critical research imperative for scalable decentralized storage.

Given the substantial size of individual files in real-world applications, auditing every data block is computationally prohibitive and economically unviable. To address this challenge, modern PDP schemes universally employ probabilistic sampling strategies. During an audit round, the verifier randomly selects a subset of data blocks and assigns independent random coefficients to construct a linear combination of challenged sectors. This approach reduces the proof size and verification complexity from linear $O(n)$ to constant or logarithmic $O(c)$ with respect to the total number of blocks, where $c \ll n$ denotes the challenge size. By carefully calibrating the number of sampled blocks, auditors can achieve arbitrarily high detection probabilities for data corruption while maintaining minimal communication and computation overhead. The randomness of the challenge set ensures that malicious providers cannot precompute proofs or selectively store only frequently accessed blocks without facing a high risk of detection.

However, the introduction of probabilistic sampling and homomorphic aggregation introduces subtle yet critical privacy vulnerabilities. In many existing batch auditing designs, challenge generation mechanisms, evaluation points, and masking techniques are either oversimplified or structurally coupled across files. If challenge points are shared, folding weights are predictable, or masking scalars lack cryptographic commitment, an honest-but-curious verifier or public ledger observer can exploit the linear algebraic structure of aggregated evaluations. Through cross-round correlation analysis or linear system inversion on publicly disclosed challenge-response transcripts, adversaries may reconstruct underlying polynomial evaluations $f(r_i)$ or even infer sensitive sector values. Such privacy leakage fundamentally contradicts the confidentiality requirements of modern cloud and decentralized storage, highlighting the necessity of integrating rigorous, one-time pad masking with binding commitments directly into the batch verification algebraic structure to prevent statistical inference attacks.

To address the aforementioned challenges, we propose **FoldAudit**, a novel batch-verifiable PDP scheme that simultaneously achieves constant-time multi-file verification, tight cryptographic soundness, and robust evaluation privacy. FoldAudit introduces a layered algebraic folding architecture: (i) independent per-file challenge points $\{r_i\}$ to structurally isolate file-specific quotient polynomials and prevent cross-file pole cancellation; (ii) a Fiat-Shamir derived random folding scalar ρ applied to quotient commitments after they are fixed, ensuring that batch aggregation remains algebraically unpredictable and bias-resistant; and (iii) fresh one-time pad masking coupled with group commitments to statistically hide challenged evaluations and point evaluations from curious verifiers. By leveraging the Zero Multiplicity Distance Lemma, we formally derive a tight soundness error bound $\epsilon \leq \frac{s+m-1}{p-m}$, guaranteeing that any adversarial error cancellation or adaptive forgery is detected with overwhelming probability. Theoretical analysis and experimental evaluations demonstrate that FoldAudit reduces on-chain verification to a constant number of pairings independent of batch size, supports stateless dynamic file updates without global state recomputation, and maintains provable privacy against honest-but-curious auditors, making it highly suitable for large-scale, privacy-sensitive decentralized storage environments.

II. PRELIMINARIES

A. Notations

For clarity and reference, we summarize the primary mathematical symbols, cryptographic parameters, and protocol-specific variables used throughout the paper in Table I.

B. Basics of Polynomials

Let $\mathbb{F}_p$ denote a finite field of prime order p , where p is determined by a security parameter λ . For a positive integer d , let $\mathbb{F}_p^{<d}[X]$ denote the vector space of univariate polynomials over $\mathbb{F}_p$ with degree strictly less than d . Polynomial indeterminates are denoted by uppercase letters (e.g., X), while field elements and coefficients use lowercase letters (e.g., $x, z, r \in \mathbb{F}_p$).

C. KZG Polynomial Commitment

We employ a KZG-type polynomial commitment scheme over a bilinear group tuple $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ of prime order p . Here, $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is an efficiently computable, non-degenerate bilinear map. Let $g \in \mathbb{G}_1$ and $h \in \mathbb{G}_2$ be publicly known generators.

A trusted setup generates a secret trapdoor $\tau \in \mathbb{F}_p$ and publishes a structured reference string (SRS) supporting polynomials up to degree $s-1$:

$$\text{SRS} = (g, g^\tau, g^{\tau^2}, \dots, g^{\tau^{s-1}}, h, h^\tau).$$

For a polynomial $f(X) = \sum_{k=0}^{s-1} a_k X^k \in \mathbb{F}_p^{<s}[X]$, its commitment is computed in the exponent as

$$C_f := g^{f(\tau)} = \prod_{k=0}^{s-1} (g^{\tau^k})^{a_k} \in \mathbb{G}_1 \quad (1)$$

Table I: Summary of Notation and System Parameters

Symbol	Description
λ	Security parameter.
p	Prime order of the finite field $\mathbb{F}_p$ and bilinear groups $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$.
$[m]$	Index set $\{1, 2, \dots, m\}$ for the audited file batch.
$\mathbb{F}_p^{<d}[X]$	Vector space of univariate polynomials over $\mathbb{F}_p$ with degree strictly less than d .
g, h	Public generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively.
$e(\cdot, \cdot)$	Efficiently computable, non-degenerate bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$.
τ	Secret trapdoor used for Structured Reference String (SRS) generation (held by DO).
SRS	Public parameters $(g, g^\tau, \dots, g^{\tau^{s-1}}, h, h^\tau)$ supporting polynomial degrees $< s$.
H_1	Collision-resistant hash function for Merkle tree construction.
H_2	Fiat-Shamir random oracle for transcript binding and deterministic challenge derivation.
m, n, s, c	Number of files, chunks per file, sectors per chunk, and challenged chunks per file.
i, j, k	Indices for files ($i \in [m]$), chunks ($j \in [n]$), and sectors ($0 \leq k < s$).
$m_{i,j,k}$	k -th sector value of chunk j in file i .
$f_{i,j}(X)$	Polynomial encoding of chunk j in file i : $f_{i,j}(X) = \sum_{k=0}^{s-1} m_{i,j,k} X^k$.
$\sigma_{i,j}$	KZG homomorphic tag for $f_{i,j}(X)$: $\sigma_{i,j} = g^{f_{i,j}(\tau)} \in \mathbb{G}_1$.
Γ_i	Root hash of the Merkle tree for file i .
J	Challenged chunk index set, $J \subseteq [n]$ with $ J = c$.
v_j	Short for $v_{i,j}$, random coefficient for challenged chunk j of file i .
r_i	Random evaluation point for file i (used for per-file quotient construction).
$f_i(X)$	Aggregated polynomial for file i : $f_i(X) = \sum_{j \in J} v_j f_{i,j}(X)$.
y_i	Evaluation claim for file i : $y_i = f_i(r_i)$.
b_i	Aggregated homomorphic tag: $b_i = \prod_{j \in J} \sigma_{i,j}^{v_j} = g^{f_i(\tau)}$.
μ_i, ν_i, η	Fresh additive masking scalars for per-file evaluations, point- z evaluations, and global privacy.
$\tilde{y}_i, \tilde{v}_{i,z}, \tilde{V}_W$	Masked evaluations: $\tilde{y}_i = y_i + \mu_i$, $\tilde{v}_{i,z} = f_i(z) + \nu_i$, $\tilde{V}_W = V_W + \eta$.
R_i, S_i, R_W	Masking commitments: $R_i = g^{\mu_i}$, $S_i = g^{\nu_i}$, $R_W = g^\eta$.
$w_i(X)$	Per-file quotient polynomial: $w_i(X) = (f_i(X) - y_i)/(X - r_i)$.
C_{w_i}	Commitment to per-file quotient: $C_{w_i} = g^{w_i(\tau)}$.
ρ	Fiat-Shamir derived folding scalar for batch quotient-sum (sampled after $\{C_{w_i}\}$ are fixed).
$W(X)$	Batch quotient-sum polynomial: $W(X) = \sum_{i=1}^m \rho^{i-1} w_i(X)$.
C_W	Commitment to batch quotient-sum: $C_W = \prod_{i=1}^m C_{w_i}^{\rho^{i-1}} = g^{W(\tau)}$.
z	Random evaluation point for batch opening (Fiat-Shamir derived, $z \notin \{r_i\}_{i=1}^m$).
V_W	True evaluation of batch polynomial: $V_W = W(z)$.
π_{batch}	KZG batch opening proof: $g^{(W(\tau) - V_W)/(\tau - z)}$.
chal	Challenge tuple $(J, \{v_j\}_{j \in J}, \{r_i\}_{i=1}^m, \text{nonce})$.
π	Complete audit proof containing tags, commitments, masked values, and π_{batch} .

To prove an evaluation $f(z) = v$, the prover computes the quotient commitment $\pi := g^{w(\tau)}$ where $w(X) = (f(X) - v)/(X - z)$. The verifier checks the pairing equation

$$e(C_f \cdot g^{-v}, h) \stackrel{?}{=} e(\pi, h^{\tau-z}), \quad (2)$$

which holds if and only if $f(\tau) - v = (\tau - z)w(\tau)$ and $w(X)$ is the quotient polynomial. Under the q -SDH assumption [], the scheme is evaluation-binding and knowledge-sound.

D. Multi-Polynomial Batch Evaluation

We present the algebraic foundation and the resulting interactive protocol for verifying multiple polynomial evaluations simultaneously.

Lemma 1 (Zero Multiplicity Distance Lemma). *Consider pairs of polynomials $(f_1, v_1), \dots, (f_m, v_m) \in \mathbb{F}_p[X]$, where the root sets $\text{zeros}(v_i) := \{z \in \mathbb{F}_p \mid v_i(z) = 0\}$ are pairwise disjoint. Define the rational function:*

$$w(X) := \sum_{i=1}^m \frac{f_i(X)}{v_i(X)},$$

and let $k_1 := \max_i (\deg(f_i) - \deg(v_i))$ and $k_2 := \sum_i \deg(v_i)$. The Zero Multiplicity Distance Lemma states that if the relative Hamming distance between w and the space of degree- $\leq k_1$ polynomials satisfies

$$\Delta(w, \mathbb{F}_{p \leq k_1}[X]) \leq 1 - \frac{k_1 + k_2 + 1}{|\mathbb{F}_p| - |Z|},$$

where $Z := \bigcup_i \text{zeros}(v_i)$, then for every i , the zero multiplicities of v_i are contained in those of f_i (denoted $\text{zmult}(v_i) \subseteq \text{zmult}(f_i)$). Intuitively, this means each denominator $v_i(X)$ must exactly divide its numerator $f_i(X)$ at all roots (counting multiplicity). If any division fails, $w(X)$ develops poles or non-polynomial behavior near the roots of v_i , forcing it to deviate significantly from any low-degree polynomial. Consequently, a single random evaluation of w will detect such algebraic inconsistencies with high probability.

Application to Multi-Polynomial Batch Evaluation. Lemma 1 provides a versatile algebraic backbone that naturally supports a two-level folding structure. Let $\rho_2 \in \mathbb{F}_p$ compress multiple polynomials evaluated at the same point, and let $\rho_1 \in \mathbb{F}_p$ fold quotient checks across distinct evaluation points. The general batch quotient-sum polynomial can be expressed as:

$$W(X) := \sum_{x \in S} \rho_1^{\text{idx}(x)-1} \left(\sum_{i=1}^{\ell_x} \rho_2^{i-1} \cdot \frac{p_{x,i}(X) - y_{x,i}}{X - x} \right),$$

where $\text{idx}(x)$ maps each point x to a unique batch index. This structure simultaneously handles intra-point polynomial compression (via ρ_2) and inter-point batch aggregation (via ρ_1).

In our Batch-PDP scenario, however, each challenge point r_i corresponds to exactly one aggregated file polynomial $f_i(X)$ (i.e., $\ell_{r_i} = 1$ for all $i \in [m]$). Consequently, the inner summation trivially collapses to a single term, rendering ρ_2 redundant. We retain only the cross-file folding scalar $\rho := \rho_1$, which is cryptographically essential for preventing adversarial error cancellation across distinct files. The polynomial simplifies to:

$$W(X) := \sum_{i=1}^m \rho^{i-1} \cdot \frac{f_i(X) - y_i}{X - r_i}.$$

The verifier samples $z \in \mathbb{F}_p \setminus \{r_i\}_{i=1}^m$ and accepts iff the evaluated sum matches the prover's committed opening $V_W = W(z)$. The consistency check directly instantiates Equation: $V_W \stackrel{?}{=} \sum_{i=1}^m \rho^{i-1} \frac{f_i(z) - y_i}{z - r_i}$. By Lemma 1 if any claim $f_i(r_i) = y_i$ is false, $W(X)$ develops a pole at r_i and deviates significantly from any degree- $(k_1 - 2)$ polynomial.

E. Merkle Tree

A Merkle tree is a binary hash-based commitment structure that provides authenticated membership proofs for a set of values. Let $H_1 : \{0, 1\}^* \rightarrow \mathbb{F}_p$ be a collision-resistant cryptographic hash function. For a sequence of n data items $\{d_j\}_{j=1}^n$, the leaves are constructed as indexed hashes: $L_j := H_1(j \parallel d_j)$, $j \in [n]$. Internal nodes are recursively computed as $N_k = H_1(N_{2k} \parallel N_{2k+1})$ until a single root hash Γ is obtained. The root Γ uniquely commits to the entire dataset under the collision resistance of H_1 . To prove that a value d_j belongs to the committed set, the prover provides an *authentication path* consisting of the sibling hashes along the route from leaf L_j to the root. The verifier recomputes the root from the path and checks equality with the trusted root Γ . Apparently, each authentication path requires $\log n$ field $\mathbb{F}_p$ elements.

III. SYSTEM MODEL AND SECURITY MODEL

A. System Model

a) System Entities: We consider a batch-verifiable Provable Data Possession (PDP) architecture comprising three entities: Data Owner, a Cloud Server and Verifier.

- **Data Owner (DO):** The DO outsources a batch of m files to the CS for long-term storage. To ensure data integrity without maintaining local copies, the DO delegates periodic auditing to the V.
- **Cloud Server (CS):** The CS is modeled as untrusted and computationally bounded: it may delete, corrupt, or selectively withhold files or chunks to save storage or bandwidth.
- **Verifier (V):** The V is honest-but-curious. It executes the auditing protocol faithfully but may attempt to infer sensitive information from audit transcripts.

b) File Representation: A file is divided into multiple chunks, and each **chunk** can be represented through polynomial coefficients. Each coefficient is referred to as a **sector**. Let m denote the number of files in a single audit batch, n the number of chunks per file, s the number of sectors per chunk. Namely, each chunk is parsed into s field elements from $\mathbb{F}_p$ and algebraically encoded as a polynomial of degree at most $s - 1$:

$$f_{i,j}(X) = \sum_{k=0}^{s-1} m_{i,j,k} X^k, \quad i \in [m], j \in [n],$$

where $m_{i,j,k}$ represents the k -th sector of the j -th chunk in file i . A **tag** is used to represent the j -th chunk through a KZG polynomial commitment. i.e., $\sigma_{i,j} = g^{f_{i,j}(\tau)} \in \mathbb{G}_1$ by Equation (1). Further, tags of each file are organized into a Merkle Tree to guarantee correct indexing and membership.

c) Protocol Workflow: The provable data possession scheme for multiple files operates through five polynomial-time algorithms:

- **Setup(λ) $\rightarrow$ pp:** Generates public parameters pp, including bilinear group tuples $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$, a structured reference string (SRS) for degree $s - 1$, and cryptographic hash functions H_1 (for Merkle Tree) and H_2, H_3 (for Fiat-Shamir transcript hashing).

- $\text{Store}(\{F_i\}_{i=1}^m) \rightarrow (\text{data}, \text{meta})$: The DO encodes each chunk into $f_{i,j}(X)$, computes tags $\sigma_{i,j}$, constructs the Merkle Tree to obtain the root hash Γ_i for each file, and outsources $\{F_i, \{\sigma_{i,j}\}, \text{Merkle trees}\}$ to the CS. The DO securely records $\{\Gamma_i\}_{i=1}^m$ and shares them with V.
- $\text{Challenge}(m, n, c) \rightarrow \text{chal}$: The verifier samples a shared challenge index set $J \subseteq [n]$ with $|J| = c$, random aggregation coefficients $\{v_{i,j}\}_{i \in [m], j \in J} \leftarrow \mathbb{F}_p$, and per-file evaluation points $\{r_i\}_{i=1}^m \leftarrow \mathbb{F}_p$. For brevity, we omit the index i in $v_{i,j}$ and simply write v_j to denote $v_{i,j}$. The challenge is $\text{chal} = (J, \{v_j\}, \{r_i\}, \text{nonce})$, where nonce is a random value to resist replay attack.
- $\text{ProofGen}(\text{chal}, \text{data}, \text{meta}) \rightarrow \pi$: The CS aggregates challenged chunks J and outputs the corresponding non-interactive proofs π .
- $\text{Verify}(\text{chal}, \pi, \{\Gamma_i\}, \text{pp}) \rightarrow \{accept, reject\}$: V validates the integrity of all files $[m]$ via the proofs π .

B. Security Model

The security of the proposed batch-PDP scheme is formalized through three core properties: correctness, soundness, and privacy.

Correctness: If all parties (DO, CS, and V) execute the protocol honestly according to the specified algorithms, the Verify algorithm outputs *accept* with probability 1.

Soundness: Let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary controlling the CS. $\mathcal{A}$ wins the soundness game if it can produce a proof π that causes Verify to output *accept* for a challenge involving corrupted, modified, or unavailable data chunks with non-negligible probability.

Privacy against Honest-but-Curious Verifier: Given the public audit transcript and the generated proof π , a PPT verifier cannot recover the challenged evaluations $\{y_i = f_i(r_i)\}_{i=1}^m$ or the underlying raw data sectors with non-negligible advantage.

IV. PROPOSED SCHEME

A. Construction

a) Setup: The Setup algorithm initializes the cryptographic environment and publishes the system-wide public parameters pp . It first generates a bilinear group tuple $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ of prime order p , selects public generators $g \in \mathbb{G}_1$ and $h \in \mathbb{G}_2$, and samples a secret trapdoor $\tau \in \mathbb{F}_p$. Using τ , the algorithm constructs the structured reference string (SRS) $(g, g^\tau, \dots, g^{\tau^{s-1}}, h, h^\tau)$, which supports polynomial commitments of degree strictly less than s . Cryptographic hash functions are fixed: H_1 for Merkle Tree construction, and H_2 for Fiat-Shamir transcript binding. Finally, $\text{pp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p, g, h, \text{SRS}, H_1, H_2)$ is published. Note that each DO has its own trapdoor τ to enable batch of polynomials and guarantee privacy of DO's files.

b) Files Storing: The Store algorithm handles data preprocessing, cryptographic tagging, and metadata generation for outsourcing. For each file F_i ($i \in [m]$), the Data Owner parses every chunk $j \in [n]$ into s field elements and algebraically encodes them as a polynomial $f_{i,j}(X) = \sum_{k=0}^{s-1} m_{i,j,k} X^k$. A KZG homomorphic tag $\sigma_{i,j} = g^{f_{i,j}(\tau)} \in \mathbb{G}_1$ is computed for each chunk, providing a compact cryptographic binding to

Algorithm 1 Setup

Require: security parameter λ

Ensure: public parameters pp

- 1: Generate $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p, g, h)$ and $\text{SRS} = (g, g^\tau, \dots, g^{\tau^{s-1}}, h, h^\tau)$.
- 2: Fix cryptographic hashes H_1, H_2 .
- 3: Publish $\text{pp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p, g, h, \text{SRS}, H_1, H_2)$.

the underlying data sectors. Note that τ is chosen and known by the data owner, so the computation of tag $\sigma_{i,j}$ can be performed in constant time. Then, these tags are organized as leaves in a Merkle Tree with root Γ_i . The tree roots $\{\Gamma_i\}$ are securely retained by the DO (and later shared with the Verifier), while the raw data and tags are outsourced to the CS. This structure guarantees both data integrity and index authenticity without requiring the DO to maintain local copies.

Algorithm 2 Store

Require: files $\{F_i\}_{i=1}^m$

Ensure: outsourced data, metadata

- 1: **for** $i = 1$ to m **do**
- 2: **for** $j = 1$ to n **do**
- 3: Parse chunk j into s sectors and form $f_{i,j}(X)$.
- 4: Compute tag $\sigma_{i,j} = g^{f_{i,j}(\tau)}$.
- 5: **end for**
- 6: Build the i th Merkle Tree over leaves $\{\sigma_{i,j}\}_{j=1}^n$.
- 7: Obtain root Γ_i .
- 8: **end for**
- 9: DO sends $(F_i, \{\sigma_{i,j}\})$ to CS and records $\{\Gamma_i\}$ for V.

c) Challenge Generation: The Chal algorithm initiates the audit by sampling a challenge tuple $\text{chal} = (J, \{v_j\}_{j \in J}, \{r_i\}_{i=1}^m)$. An index set $J \subseteq [n]$ of size c for each file is selected to query the chunk positions. Random coefficients $\{v_j\}$ (which is short for $v_{i,j}$) are drawn uniformly from $\mathbb{F}_p$ to enable random linear aggregation of challenged chunks; this masks individual chunk values, prevents targeted corruption, and enables homomorphic tag recombination. Additionally, independent per-file evaluation points $\{r_i\}$ are sampled to support file-specific quotient consistency checks. The challenge is transmitted to the CS alongside a fresh *nonce* to prevent replay or precomputation attacks.

Algorithm 3 Challenge

Require: parameters (m, n, c)

Ensure: challenge chal

- 1: Sample one shared challenged index set $J \subseteq [n]$, $|J| = c$.
- 2: Sample random $\{v_j \xleftarrow{\$} \mathbb{F}_p\}_{j \in J}$.
- 3: Sample random $\{r_i \xleftarrow{\$} \mathbb{F}_p\}_{i=1}^m$.
- 4: Set $\text{chal} = (J, \{v_j\}_{j \in J}, \{r_i\}_{i=1}^m, \text{nonce})$.

d) Proof Generation: Upon receiving the challenge $\text{chal} = (J, \{v_j\}_{j \in J}, \{r_i\}_{i=1}^m, \text{nonce})$, the ProofGen algorithm by the CS computes a compact, cross-file aggregated proof.

Algorithm 4 ProofGen

Require: challenge $\text{chal} = (J, \{v_{i,j}\}_{i \in [m], j \in J}, \{r_i\}_{i=1}^m, \text{nonce})$, files $\{F_i\}_{i=1}^m$, tags $\{\sigma_{i,j}\}$

Ensure: proof π

- 1: **for** $i = 1$ to m **do**
 Step 1: Per-File Tag Authentication Paths Building
- 2: Build Merkle paths Auth_i with $\{\sigma_{i,j}\}_{j \in J}$ for file i
 Step 2: Per-File Polynomial Aggregation and Quotient Commitment with Privacy Masking
- 3: $f_i(X) \leftarrow \sum_{j \in J} v_{i,j} f_{i,j}(X)$
- 4: $y_i \leftarrow f_i(r_i)$
- 5: Sample $\mu_i \xleftarrow{\$} \mathbb{F}_p$, set $\tilde{y}_i \leftarrow y_i + \mu_i$, $R_i \leftarrow g^{\mu_i}$
- 6: $w_i(X) \leftarrow \frac{f_i(X) - y_i}{X - r_i} \triangleright$ Polynomial division over $\mathbb{F}_p$
- 7: $C_{w_i} \leftarrow g^{w_i(\tau)}$
- 8: **end for**
 Step 2: Cross-File Quotient Folding and Masked Batch Evaluation
- 9: $b = g^{\sum_{i=1}^m f_i(\tau)} \triangleright$ Randomly combined tags
- 10: $\rho \leftarrow H_2(\text{chal}, \{\Gamma_i\}, b, \{\tilde{y}_i\}, \{R_i\}, \{C_{w_i}\}, "1")$
- 11: $W(X) \leftarrow \sum_{i=1}^m \rho^{i-1} w_i(X)$
- 12: $z \leftarrow H_2(\text{chal}, \{\Gamma_i\}, b, \{\tilde{y}_i\}, \{R_i\}, \{C_{w_i}\}, "2") \neq r_i$
- 13: $V_W \leftarrow W(z) = \sum_{i=1}^m \rho^{i-1} \cdot \frac{f_i(z) - y_i}{z - r_i}$
- 14: **for** $i = 1$ to m **do**
- 15: $v_{i,z} \leftarrow f_i(z)$
- 16: Sample $\nu_i \xleftarrow{\$} \mathbb{F}_p$, set $\tilde{v}_{i,z} \leftarrow v_{i,z} + \nu_i$, $S_i \leftarrow g^{\nu_i}$
- 17: **end for**
- 18: Sample $\eta \xleftarrow{\$} \mathbb{F}_p$, set $\tilde{V}_W \leftarrow V_W + \eta$, $R_W \leftarrow g^\eta$
 Step 3: Batch KZG Opening Proof Generation
- 19: $\pi_{\text{batch}} \leftarrow g^{\phi(\tau)}$, where $\phi(X) = \frac{W(X) - V_W}{X - z}$
- 20: **return** $\pi = (\{\text{Auth}_i, \tilde{y}_i, R_i, C_{w_i}, \{\sigma_{i,j}\}_{j \in J}\}_{i=1}^m, b, \tilde{V}_W, R_W, \pi_{\text{batch}}, \{\tilde{v}_{i,z}, S_i\}_{i=1}^m)$

- 1) *Step 1: Per-File Tag Authentication Paths Building.* For each file $i \in [m]$, constructs the Merkle paths Auth_i using the tags $\{\sigma_{i,j}\}_{j \in J}$.
- 2) *Step 2: Per-File Polynomial Aggregation and Quotient Commitment with Privacy Masking.* For each file $i \in [m]$, the CS homomorphically aggregates the challenged chunks into a single polynomial $f_i(X) = \sum_{j \in J} v_{i,j} f_{i,j}(X)$. It evaluates f_i at the per-file challenge point r_i to obtain $y_i = f_i(r_i)$. To prevent leakage of y_i to an honest-but-curious verifier, the CS samples a fresh mask $\mu_i \xleftarrow{\$} \mathbb{F}_p$, computes the masked evaluation $\tilde{y}_i = y_i + \mu_i$, and commits to the mask as $R_i = g^{\mu_i}$. Subsequently, it constructs the per-file quotient polynomial $w_i(X) = \frac{f_i(X) - y_i}{X - r_i}$. Since $f_i(r_i) = y_i$, $w_i(X)$ is a well-defined polynomial of degree at most $s - 2$. The CS commits to it as $C_{w_i} = g^{w_i(\tau)}$.
- 3) *Step 3: Cross-File Quotient Folding and Masked Batch Evaluation.* Further, b the challenged cross-file tags are combined as $b = g^{\sum_{i=1}^m f_i(\tau)}$, where the sum polynomial $\sum_{i=1}^m f_i(X)$ can be evaluated. To compress m independent quotient checks into a single constant-size verification, the CS derives a folding scalar $\rho \leftarrow H_2(\dots, "1")$. This binding ensures ρ is unpredictable and fixed after all per-file quotient commitments are published. The

Algorithm 5 Verify

Require: challenge chal , proof π , Merkle roots $\{\Gamma_i\}_{i=1}^m$, public parameters pp

Ensure: accept or reject

- 1: Parse π into $(\{\text{Auth}_i, \tilde{y}_i, R_i, C_{w_i}, \{\sigma_{i,j}\}_{j \in J}\}_{i=1}^m, b, \tilde{V}_W, R_W, \pi_{\text{batch}}, \{\tilde{v}_{i,z}, S_i\}_{i=1}^m)$
 Step 1: Tag Authentication
- 2: **for** $i = 1$ to m **do**
- 3: Verify Merkle paths Auth_i against $(\Gamma_i, \{\sigma_{i,j}\}_{j \in J})$
- 4: **if** verification fails **then return reject**
- 5: **end if**
- 6: **end for**
 Step 2: Tag Consistency Pre-checks
- 7: **if** $b \neq \prod_{i=1}^m \prod_{j \in J} \sigma_{i,j}^{v_{i,j}}$ **then return reject**
- 8: **end if**
 Step 3: Cross-File Quotient Folding Commitment and Batch KZG Opening Verification
- 9: $\rho \leftarrow H_2(\text{chal}, \{\Gamma_i\}, b, \{\tilde{y}_i\}, \{R_i\}, \{C_{w_i}\}, "1")$
- 10: $C_W \leftarrow \prod_{i=1}^m C_{w_i}^{\rho^{i-1}}$
- 11: $z \leftarrow H_2(\text{chal}, \{\Gamma_i\}, b, \{\tilde{y}_i\}, \{R_i\}, \{C_{w_i}\}, "2")$
- 12: **if** $e(C_W \cdot g^{-\tilde{V}_W} \cdot R_W \cdot \pi_{\text{batch}}^z, h) \neq e(\pi_{\text{batch}}, h^\tau)$ **then return reject**
- 13: **end if**
 Step 4: Cross-File Evaluation Consistency
- 14: Compute LHS $\leftarrow g^{\tilde{V}_W} \cdot R_W^{-1} \cdot g^{-\sum_{i=1}^m \frac{\rho^{i-1}(\tilde{v}_{i,z} - \tilde{y}_i)}{z - r_i}}$
- 15: Compute RHS $\leftarrow \prod_{i=1}^m (R_i \cdot S_i^{-1})^{\frac{\rho^{i-1}}{z - r_i}}$
- 16: **if** LHS $\neq$ RHS **then return reject**
- 17: **end if**
- 18: **return accept**

CS then constructs the batch quotient-sum polynomial $W(X) = \sum_{i=1}^m \rho^{i-1} w_i(X)$. Then, the CS samples a random evaluation point $z \leftarrow H_2(\dots, "2")$. It evaluates each aggregated file polynomial at z as $v_{i,z} = f_i(z)$, applies per-file masks $\nu_i \xleftarrow{\$} \mathbb{F}_p$ to obtain $\tilde{v}_{i,z} = v_{i,z} + \nu_i$ with commitments $S_i = g^{\nu_i}$, and applies a global mask $\eta \xleftarrow{\$} \mathbb{F}_p$ to $V_W = W(z)$ to yield $\tilde{V}_W = V_W + \eta$ with commitment $R_W = g^\eta$.

- 4) *Step 4: Random Evaluation & KZG Opening and Masking.* Finally, the CS computes the KZG opening proof $\pi_{\text{batch}} = g^{\phi(\tau)}$ for $\phi(X) = \frac{W(X) - V_W}{X - z}$ and outputs the complete proof $\pi = (\{\text{Auth}_i, \tilde{y}_i, R_i, C_{w_i}, \{\sigma_{i,j}\}_{j \in J}\}_{i=1}^m, b, \tilde{V}_W, R_W, \pi_{\text{batch}}, \{\tilde{v}_{i,z}, S_i\}_{i=1}^m)$.

e) Verification: The Verify algorithm validates the proof through a sequence of deterministic checks that require only a constant number of pairings, independent of the batch size m .

- 1) *Step 1: Tag Authentication.* The verifier first validates each Auth_i against the trusted root Γ_i to guarantee index authenticity under the collision resistance of H_1 .
- 2) *Step 2: Tag Consistency Pre-checks.* The verifier then recomputes the aggregated tag $\prod_{j \in J} \sigma_{i,j}^{v_{i,j}}$ and rejects if the result is not equal to b , cryptographically binding the tags to the legitimate challenged polynomials.

- 3) *Step 3: Cross-File Quotient Folding Commitment and Batch KZG Opening Verification.* Next, the verifier independently derives ρ and z via H_2 . It computes the batch commitment $C_W = \prod_{i=1}^m C_{w_i}^{\rho^{i-1}}$. To verify that $W(X)$ is indeed a low-degree polynomial committing to V_W , the verifier checks the KZG opening equation with statistical masks: $e\left(C_W \cdot g^{-\tilde{V}_W} \cdot R_W \cdot \pi_{\text{batch}}^z, h\right) \stackrel{?}{=} e(\pi_{\text{batch}}, h^\tau)$. This pairing check certifies that $W(\tau) - \tilde{V}_W + \eta = (\tau - z)\phi(\tau)$, ensuring algebraic consistency of the folded quotient-sum.
- 4) *Step 4: Cross-File Evaluation Consistency.* Instead of verifying each per-file quotient $w_i(X)$ individually, the verifier enforces the folded consistency relation across all files via a single scalar equation in $\mathbb{G}_1$. The target algebraic identity (in the exponent) is $V_W \stackrel{?}{=} \sum_{i=1}^m \rho^{i-1} \frac{v_{i,z} - y_i}{z - r_i}$, where $v_{i,z} = f_i(z)$, $y_i = f_i(r_i)$, and $V_W = W(z)$. To preserve privacy against an honest-but-curious verifier, all field elements are statistically masked: $\tilde{y}_i = y_i + \mu_i$, $\tilde{v}_{i,z} = v_{i,z} + \nu_i$, and $\tilde{V}_W = V_W + \eta$, with corresponding commitments $R_i = g^{\mu_i}$, $S_i = g^{\nu_i}$, and $R_W = g^\eta$. The verifier translates the masked identity into a group equality by isolating all committed masks on one side and all masked evaluations on the other. Specifically, it computes:

$$\begin{aligned} \text{LHS} &:= g^{\tilde{V}_W} \cdot R_W^{-1} \cdot \prod_{i=1}^m \left(g^{\tilde{y}_i} \cdot g^{-\tilde{v}_{i,z}} \right)^{\frac{\rho^{i-1}}{z - r_i}} \\ &= g^{\tilde{V}_W} \cdot R_W^{-1} \cdot g^{-\sum_{i=1}^m \frac{\rho^{i-1}(\tilde{v}_{i,z} - \tilde{y}_i)}{z - r_i}} \\ \text{RHS} &:= \prod_{i=1}^m \left(R_i \cdot S_i^{-1} \right)^{\frac{\rho^{i-1}}{z - r_i}}. \end{aligned}$$

The verifier accepts iff $\text{LHS} \stackrel{?}{=} \text{RHS}$. By Lemma 1, successful validation implies that all per-file quotients $w_i(X)$ are algebraically valid, rendering bias or pole-cancellation attacks impossible.

If all checks pass, the verifier outputs *accept*; otherwise, it outputs *reject*.

B. Security Analysis

We formally establish the correctness, soundness, and privacy of the proposed batch-PDP scheme. The security analysis relies on the following standard cryptographic assumptions:

Assumption 1 (Collision-Resistant Hash). *The hash functions H_1 (for Merkle tree construction) and H_2 (for Fiat-Shamir transcript binding) are collision-resistant. No PPT adversary can find distinct inputs $x \neq x'$ such that $H_1(x) = H_1(x')$ or $H_2(x) = H_2(x')$ with non-negligible probability.*

Assumption 2 (KZG Knowledge Soundness). *The KZG polynomial commitment scheme over $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ is evaluation-binding and knowledge-sound under the q -SDH assumption. Specifically, for any PPT adversary producing a valid opening proof π for commitment $C = g^{f(\tau)}$ at point z with value v , there exists a PPT extractor that outputs the unique polynomial $f(X) \in \mathbb{F}_{<s}[X]$ satisfying $f(z) = v$.*

Assumption 3 (Discrete Logarithm Hardness). *The discrete logarithm problem in $\mathbb{G}_1$ is computationally intractable. Given $g \in \mathbb{G}_1$ and $Y = g^y$, no PPT adversary can recover $y \in \mathbb{F}_p$ with non-negligible advantage.*

Theorem 1 (Correctness). *If the Data Owner, Cloud Server, and Verifier execute Algorithms 2, 3, 4, and 5 honestly, then Verify outputs accept with probability 1.*

Proof. For each file $i \in [m]$, the server aggregates the challenged chunks into $f_i(X) = \sum_{j \in J} v_{j,i} f_{i,j}(X)$ and computes $y_i = f_i(r_i)$. By construction, $f_i(r_i) - y_i = 0$, so $(X - r_i)$ divides $f_i(X) - y_i$ exactly. Thus, the per-file quotient $w_i(X) = \frac{f_i(X) - y_i}{X - r_i}$ is a well-defined polynomial over $\mathbb{F}_p$ with $\deg(w_i) \leq s - 2$. The batch quotient-sum polynomial $W(X) = \sum_{i=1}^m \rho^{i-1} w_i(X)$ consequently satisfies $\deg(W) \leq s - 2$, ensuring all associated commitments lie within the SRS degree bound.

Step 1: Tag Authentication. The Merkle authentication paths Auth_i are constructed honestly from the leaves $\{\sigma_{i,j}\}_{j \in J}$, so they verify against the trusted root Γ_i with probability 1 under the collision resistance of H_1 .

Step 2: Tag Consistency Pre-checks. For tag recombination, the verifier verifies:

$$\begin{aligned} b &= \prod_{i=1}^m g^{f_i(\tau)} = \prod_{i=1}^m g^{\sum_{j \in J} v_{j,i} f_{i,j}(\tau)} \\ &= \prod_{j \in J} \left(g^{f_{i,j}(\tau)} \right)^{v_j} = \prod_{i=1}^m \prod_{j \in J} \sigma_{i,j}^{v_j} = g^{f_i(\tau)}. \end{aligned}$$

Step 3: Cross-File Quotient Folding Commitment and Batch KZG Opening Verification. KZG batch proof for quotient folding polynomial $W(X)$ is $\pi_{\text{batch}} = g^{\frac{W(\tau) - V_W}{\tau - z}}$, where $V_W = W(z)$. Fresh one-time masks are applied: $\tilde{V}_W = V_W + \eta$ and $R_W = g^\eta$. Substituting these into the pairing check yields:

$$\begin{aligned} &e(C_W \cdot g^{-\tilde{V}_W} \cdot R_W \cdot \pi_{\text{batch}}^z, h) \\ &= e(g^{W(\tau)} \cdot g^{-(V_W + \eta)} \cdot g^\eta \cdot g^{z \cdot \frac{W(\tau) - V_W}{\tau - z}}, h) \\ &= e(g^{W(\tau) - V_W - \eta + \eta + z \cdot \frac{W(\tau) - V_W}{\tau - z}}, h) \\ &= e(g^{(W(\tau) - V_W)(1 + \frac{z}{\tau - z})}, h) \\ &= e(g^{(W(\tau) - V_W) \frac{\tau}{\tau - z}}, h). \\ &= e(\pi_{\text{batch}}, h^\tau) \end{aligned}$$

Step 4: Cross-File Evaluation Consistency. The verifier checks the scalar relation:

$$\begin{aligned} \text{LHS} &= g^{\tilde{V}_W} \cdot R_W^{-1} \cdot g^{-\sum_{i=1}^m \frac{\rho^{i-1}(\tilde{v}_{i,z} - \tilde{y}_i)}{z - r_i}} \\ &= g^{V_W + \eta} \cdot g^{-\eta} \cdot g^{-\sum_{i=1}^m \frac{\rho^{i-1}((v_{i,z} + \nu_i) - (y_i + \mu_i))}{z - r_i}} \\ &= g^{V_W} \cdot g^{-\sum_{i=1}^m \frac{\rho^{i-1}(v_{i,z} - y_i)}{z - r_i}} \cdot g^{\sum_{i=1}^m \frac{\rho^{i-1}(\mu_i - \nu_i)}{z - r_i}} \\ &= \left(g^{V_W - \sum_{i=1}^m \frac{\rho^{i-1}(v_{i,z} - y_i)}{z - r_i}} \right) \cdot \left(\prod_{i=1}^m g^{(\mu_i - \nu_i) \frac{\rho^{i-1}}{z - r_i}} \right) \\ &= g^{V_W - V_W} \cdot \left(\prod_{i=1}^m (R_i \cdot S_i^{-1})^{\frac{\rho^{i-1}}{z - r_i}} \right) \\ &= \text{RHS} \end{aligned}$$

Since all verification checks (Merkle authentication, tag combination, KZG batch opening, and cross-file consistency) are satisfied with probability 1 under honest execution, the Verify algorithm outputs *accept*. $\square$

Theorem 2 (Soundness). *Let $\mathcal{A}$ be a PPT adversary controlling the Cloud Server. Under Assumptions 1–3 and modeling H_2 as a random oracle, if $\mathcal{A}$ outputs a proof π that causes Verify to accept for a challenge involving corrupted or unavailable data, then $\mathcal{A}$ succeeds with probability at most $\frac{s+m-1}{p-m} + \text{negl}(\lambda)$.*

Proof. We prove soundness via a sequence of hybrid games. Let Win_k denote the event that the PPT adversary $\mathcal{A}$ controlling the Cloud Server causes Verify to accept for a challenge involving corrupted or unavailable data in Game k .

Game 0: Real Soundness Game. Identical to the real protocol execution. By definition, $\Pr[\text{Win}_0] = \epsilon$.

Game 1: Index Authenticity & Tag Binding. The challenger strictly enforces Merkle path verification against the trusted roots $\{\Gamma_i\}_{i=1}^m$ and verifies the tag recombination $\prod_{j \in J} \sigma_{i,j}^{v_j} \stackrel{?}{=} b_i$. The Merkle verification relies solely on the collision resistance of H_1 . The tag binding relies on the homomorphic property of KZG commitments and the discrete logarithm assumption in $\mathbb{G}_1$. Any successful deviation by $\mathcal{A}$ implies breaking H_1 or forging a KZG tag, both of which occur with negligible probability. Thus, $|\Pr[\text{Win}_0] - \Pr[\text{Win}_1]| \leq \text{negl}(\lambda)$.

Game 2: Quotient-Consistency Enforcement. Identical to Game 1, except the challenger strictly enforces the batch KZG opening verification equation from Algorithm 5, Step 2. Under Assumption 2 and Assumption 3, a PPT adversary cannot produce a valid proof tuple $(C_W, \tilde{V}_W, R_W, \pi_{\text{batch}})$ that passes this pairing check unless the committed polynomial $W(X)$ and its claimed evaluation V_W are cryptographically bound to a well-defined quotient structure. Concretely, the check forces the existence of polynomials $\{w_i(X)\}_{i=1}^m$ such that $W(X) = \sum_{i=1}^m \rho^{i-1} w_i(X)$ and $w_i(\tau) = (f_i(\tau) - y_i) / (\tau - r_i)$ for all $i \in [m]$. Any adversary that succeeds in Game 1 but fails in Game 2 must have generated a proof that bypasses the tag/Merkle checks yet violates the KZG opening relation. Constructing such a proof implies either forging a valid KZG evaluation binding or solving the discrete logarithm problem in $\mathbb{G}_1$, both of which occur with negligible probability. Therefore, the transition between games is statistically indistinguishable to any PPT adversary: $|\Pr[\text{Win}_1] - \Pr[\text{Win}_2]| \leq \text{negl}(\lambda)$.

Game 3: Folded Evaluation Consistency Enforcement. Identical to Game 2, except the challenger explicitly verifies the algebraic identity

$$V_W \stackrel{?}{=} \sum_{i=1}^m \rho^{i-1} \frac{f_i(z) - y_i}{z - r_i}$$

at the random oracle-derived point $z \in \mathbb{F}_p \setminus \{r_i\}_{i=1}^m$. By Assumption 2, a PPT extractor $\mathcal{E}$ can retrieve a unique polynomial $\hat{W}(X) \in \mathbb{F}_p^{\leq s-2}[X]$ such that $C_W = g^{\hat{W}(\tau)}$ and $\hat{W}(z) = V_W$. The adversary $\mathcal{A}$ wins in Game 2 but loses in Game 3 only if it forges a proof where $\hat{W}(z) = h(z)$ for a

corrupted file i^* satisfying $f_{i^*}(r_{i^*}) \neq y_{i^*}$, where we define the rational function

$$h(X) := \sum_{i=1}^m \rho^{i-1} \frac{f_i(X) - y_i}{X - r_i}.$$

Mapping to the notation of Lemma 1, set $u_i(X) \leftarrow f_i(X) - y_i$ and $v_i(X) \leftarrow X - r_i$. The denominator roots $Z := \bigcup_{i=1}^m \text{zeros}(v_i) = \{r_1, \dots, r_m\}$ are pairwise disjoint with $|Z| = m$. We have $k_1 := \max_i (\deg(u_i) - \deg(v_i)) = s-2$ and $k_2 := \sum_{i=1}^m \deg(v_i) = m$. Since $(X - r_{i^*}) \nmid u_{i^*}(X)$, the zero-multiplicity condition $\text{zmult}(v_{i^*}) \sqsubseteq \text{zmult}(u_{i^*})$ is violated. By the contrapositive of Lemma 1, $h(X)$ is algebraically far from any polynomial in $\mathbb{F}_p^{\leq s-2}[X]$. Consequently, the probability that a uniformly random $z \in \mathbb{F}_p \setminus Z$ (sampled after commitments are fixed) satisfies $h(z) = \hat{W}(z)$ is bounded by the agreement set size:

$$\Pr_z [h(z) = \hat{W}(z)] \leq \frac{k_1 + k_2 + 1}{p - |Z|} = \frac{(s-2) + m + 1}{p - m} = \frac{s + m - 1}{p - m}.$$

Therefore, the game transition satisfies $|\Pr[\text{Win}_2] - \Pr[\text{Win}_3]| \leq \frac{s+m-1}{p-m}$.

Combining all game transitions yields

$$\epsilon \leq \frac{s + m - 1}{p - m} + \text{negl}(\lambda),$$

which completes the proof. $\square$

Theorem 3 (Privacy against Honest-but-Curious Verifier). *Under Assumption 3 and assuming that the masking scalars $\{\mu_i\}_{i=1}^m$, $\{\nu_i\}_{i=1}^m$, and η are freshly and uniformly sampled from $\mathbb{F}_p$ in each audit round, no PPT honest-but-curious verifier can recover the challenged evaluations $\{y_i = f_i(r_i)\}_{i=1}^m$ or the underlying raw data sectors $\{m_{i,j,k}\}$ from the audit transcript and proof π with non-negligible advantage.*

Proof. We prove privacy via a standard simulation-based argument. We construct a probabilistic polynomial-time (PPT) simulator $\mathcal{S}$ that, given only the public parameters pp , the Merkle roots $\{\Gamma_i\}_{i=1}^m$, and the challenge chal , generates a simulated proof π_{sim} that is computationally indistinguishable from a real proof π generated by an honest cloud server. If such a simulator exists, the verifier's view reveals no information about the underlying data beyond the public challenge.

Let the real transcript be $\mathcal{T} = (\text{chal}, \pi, \{\Gamma_i\}_{i=1}^m)$, where π contains the tuple $(\{\text{Auth}_i, b_i, \tilde{y}_i, R_i, C_{w_i}, \{\sigma_{i,j}\}_{j \in J}\}_{i=1}^m, \tilde{V}_W, R_W, \pi_{\text{batch}}, \{\tilde{v}_{i,z}, S_i\}_{i=1}^m)$.

The simulator $\mathcal{S}$ proceeds as follows:

- 1) $\mathcal{S}$ copies the public structural elements identically: the challenge chal , the Merkle authentication paths $\{\text{Auth}_i\}_{i=1}^m$, the aggregated tags $\{b_i\}_{i=1}^m$, the per-file and global commitments $\{C_{w_i}\}$, and the individual challenged tags $\{\sigma_{i,j}\}$. These values are either public, deterministically derived from public parameters, or cryptographically bound to file/chunk indices; they do not leak raw sector values without solving the discrete logarithm or breaking the collision resistance of H_1 .

- 2) For each file $i \in [m]$, $\mathcal{S}$ samples $\tilde{y}'_i \leftarrow \mathbb{F}_p$, $\tilde{v}'_{i,z} \leftarrow \mathbb{F}_p$, and $\tilde{V}'_W \leftarrow \mathbb{F}_p$ uniformly at random.
- 3) For each $i \in [m]$, $\mathcal{S}$ samples independent random group elements $R'_i, S'_i \leftarrow \mathbb{G}_1$ and $R'_W \leftarrow \mathbb{G}_1$.
- 4) $\mathcal{S}$ samples a random group element $\pi'_{\text{batch}} \leftarrow \mathbb{G}_1$.
- 5) $\mathcal{S}$ assembles π_{sim} using the components above and outputs $(\text{chal}, \pi_{\text{sim}}, \{\Gamma_i\}_{i=1}^m)$.

We now establish the indistinguishability between $\mathcal{T}$ and $\mathcal{T}_{\text{sim}}$ through the following properties:

- *Statistical Hiding of Evaluations.* In the real protocol, the masked evaluations are computed as $\tilde{y}_i = y_i + \mu_i$, $\tilde{v}_{i,z} = f_i(z) + \nu_i$, and $\tilde{V}_W = V_W + \eta$, where $\mu_i, \nu_i, \eta \leftarrow \mathbb{F}_p$ are freshly sampled uniform random scalars. By the one-time pad property, each masked value is statistically uniformly distributed over $\mathbb{F}_p$ and independent of the underlying true values y_i , $f_i(z)$, and V_W . Consequently, the simulator's random choices $\tilde{y}'_i, \tilde{v}'_{i,z}, \tilde{V}'_W$ are identically distributed to the real values, yielding perfect statistical hiding: $I(y_i; \tilde{y}_i) = 0$.
- *Computational Hiding of Mask Scalars.* The real masking commitments are $R_i = g^{\mu_i}$, $S_i = g^{\nu_i}$, and $R_W = g^\eta$. Under Assumption 3, no PPT adversary can recover the exponents μ_i, ν_i, η from these group elements with non-negligible probability. Since the simulated commitments R'_i, S'_i, R'_W are uniformly random elements in $\mathbb{G}_1$, they are computationally indistinguishable from the real commitments under the discrete logarithm assumption.
- *Hiding of Polynomial Commitments.* The KZG commitments $\{C_{w_i}\}$, and the opening proof π_{batch} are computed over the public SRS without revealing the trapdoor τ . By the construction of the KZG scheme, these commitments are perfectly hiding with respect to the polynomial coefficients. They only certify algebraic consistency at the queried points (which are already statistically masked). Without knowledge of τ or the underlying polynomials, these group elements leak zero information about the raw sector coefficients $\{m_{i,j,k}\}$ or the evaluation claims.

Combining the above, the joint distribution of the field elements in π is statistically independent of the underlying data, while the group elements are computationally indistinguishable from random under Assumption 3. Therefore, the real transcript $\mathcal{T}$ and the simulated transcript $\mathcal{T}_{\text{sim}}$ are computationally indistinguishable for any PPT honest-but-curious verifier. It follows that

$$\Pr[\mathcal{V} \text{ recovers } \{y_i\} \text{ or } \{m_{i,j,k}\} \mid \mathcal{T}] \leq \text{negl}(\lambda),$$

which completes the proof. $\square$

C. Complexity Analysis

We measure computational overhead in terms of group exponentiations in $\mathbb{G}_1$ (E), and bilinear pairings (P). Communication overhead is measured in bits using $|p|$, $|G_1|$, and $|h|$ to denote the bit lengths of a field element, a group element, and a cryptographic hash digest, respectively.

a) Computation Complexity.

- *Store Algorithm.* Since the DO holds the secret trapdoor τ , each homomorphic tag $\sigma_{i,j} = g^{f_{i,j}(\tau)}$ is computed via direct polynomial evaluation in $1E$. Across m files and n chunks, the total cost is mnE .
- *ProofGen Algorithm.* For each file $i \in [m]$, the CS computes the privacy commitment $R_i = g^{\mu_i}$ ($1E$), and commits to the per-file quotient polynomial C_{w_i} $((s-1)E)$. Calculating b requires sE . Per-file point evaluations and masking commitments $\{S_i\}_{i=1}^m$ cost mE . The global masking commitment R_W costs $1E$, and the KZG opening proof π_{batch} requires committing to a degree- $(s-3)$ quotient polynomial $((s-2)E)$. Summing these components, the prover's exact cryptographic workload is $(ms + m + 2s - 1)E$, which scales as $\mathcal{O}(msE)$.

$$\underbrace{msE}_{\text{Step 1}} + \underbrace{(s + m + 1)E}_{\text{Step 2}} + \underbrace{(s - 2)E}_{\text{Step 3}} = (ms + m + 2s - 1)E,$$

- *Verify Algorithm.* The verifier recomputes aggregated tags b (mcE) and the folded quotient commitment C_W (mE). The batch KZG opening check (i.e., Step 3), requiring $2E + 2P$. The cross-file consistency check (i.e., Step 4) requires $3E$ for LHS and mE for RHS. Thus, the verifier performs $(mc + 2m + 3)E + 2P$ cryptographic operations.

$$\underbrace{mcE}_{\text{Step 2}} + \underbrace{mE}_{\text{Step 3}} + \underbrace{(m + 3)E}_{\text{Step 4}} = (mc + 2m + 3)E + 2P,$$

b) Communication Complexity.

- *Store Algorithm.* The DO transmits raw file data to the CS, incurring $mns|p|$ bits. Concurrently, mn homomorphic tags $\{\sigma_{i,j}\}$ are outsourced to enable future homomorphic aggregation, costing $mn|\mathbb{G}_1|$ bits. The DO securely shares m Merkle root hashes $\{\Gamma_i\}_{i=1}^m$ with the verifier for audit authentication, requiring $m|h|$ bits. The total one-time outsourcing communication is $mns|p| + mn|\mathbb{G}_1| + m|h|$.

$$\underbrace{mns|p|}_{\text{File data}} + \underbrace{mn|\mathbb{G}_1|}_{\text{Tags}} + \underbrace{m|h|}_{\text{Merkle roots}} = mns|p| + mn|\mathbb{G}_1| + m|h|,$$

- *ProofGen Algorithm.* For each file $i \in [m]$, the proof payload includes $(\text{Auth}_i, \tilde{y}_i, R_i, C_{w_i}, \{\sigma_{i,j}\}_{j \in J})$, contributing $c|\mathbb{G}_1| + 2|\mathbb{G}_1| + |p| + c \log_2 n|h|$ per file. The cross-file aggregation includes $(b, \tilde{V}_W, R_W, \pi_{\text{batch}}, \{\tilde{v}_{i,z}, S_i\}_{i=1}^m)$, adding $3|\mathbb{G}_1| + |p| + m(|\mathbb{G}_1| + |p|)$ bits. Aggregating across all files, the total audit proof size scales as:

$$(mc + 3m + 3)|G_1| + (2m + 1)|p| + c \log_2 n|h|.$$

D. Dynamic File Management

The scheme natively supports file addition and deletion through independent per-file KZG commitments and modular Merkle trees. Crucially, ProofGen and Verify are stateless per audit round; they operate on the current active file set without maintaining persistent or incremental proof state. Dynamic operations thus only affect the Store phase and metadata synchronization.

When the DO uploads a new file F_{m+1} , it encodes each chunk into $f_{m+1,j}(X)$, computes tags $\sigma_{m+1,j} = g^{f_{m+1,j}(\tau)}$, and constructs a fresh Merkle tree yielding root Γ_{m+1} . The data and tags are outsourced to the CS, while Γ_{m+1} is securely shared with the verifier. The active batch size increments to $m \leftarrow m + 1$. Subsequent audits automatically incorporate F_{m+1} via the expanded index collection and Merkle root set.

To securely remove $F_{i_{\text{del}}}$, the CS permanently erases the associated data, tags, and Merkle nodes. The DO and V discard $\Gamma_{i_{\text{del}}}$. Surviving files simply skip the removed index in subsequent audit loops, incurring zero recomputation of tags or Merkle roots. The batch size decrements to $m \leftarrow m - 1$, and future ProofGen executions simply iterate over the reduced active set.

Complexity of Dynamic Operations. Adding or deleting a single file requires $\mathcal{O}(nE)$ time for tag generation, as implied by the Store algorithm. Importantly, no SRS regeneration, trapdoor rotation, or global commitment recomputation is needed. Since proof generation is challenge-driven and stateless, dynamic updates incur overhead strictly at the storage and metadata layers. Consequently, the ProofGen and Verify algorithms retain the same complexity as analyzed in Section IV-C.

V. RELATED WORKS

Recent years have witnessed significant advances in provable data possession (PDP) schemes for cloud and decentralized storage systems. Several blockchain-assisted auditing mechanisms have been proposed to enhance transparency and eliminate reliance on trusted third-party auditors.

Xu et al. [XuTIFS26] proposed MPA, a lightweight integrity auditing scheme integrating polynomial commitments with Merkle trees for multi-copy decentralized storage. Their design achieves efficient block-level verification and supports dynamic updates through rank-based Merkle tree structures. While MPA enables batch auditing across multiple providers and employs blockchain-based commit-and-reveal protocols for challenge generation, it utilizes a shared evaluation point across all files. In the multi-file batch auditing of XuTIFS26, the reliance on a single shared challenge point r introduces a cross-file error cancellation vulnerability. Consider an adversary targeting two audited files F_i and F_j . The adversary submits forged evaluation claims $val_i^* = f_i(r) + \delta$ and $val_j^* = f_j(r) - \delta$ for an arbitrary $\delta \in \mathbb{Z}_p^*$. Leveraging the homomorphic property of KZG commitments, the adversary constructs compensating quotient witnesses $w_i^* = g^{h_i(\alpha) - \delta / (\alpha - r)}$ and $w_j^* = g^{h_j(\alpha) + \delta / (\alpha - r)}$, where $h_k(X) = (f_k(X) - val_k^*) / (X - r)$. During verification, the global pairing check evaluates $\hat{e}(\sigma^{\text{global}} \cdot g^{-val^{\text{global}}}, g) \stackrel{?}{=} \hat{e}(w^{\text{global}}, g^{\alpha - r})$. Since the aggregate evaluation error $\sum_k (val_k^* - f_k(r))$ vanishes and the forged witnesses satisfy $\sum_k (\alpha - r) \cdot \log_g w_k^* = \sum_k (f_k(\alpha) - val_k^*)$, the verification equation holds with probability 1. Consequently, the global check accepts the batch despite individual file corruption, demonstrating that the protocol enforces only aggregate polynomial consistency rather than strict per-file soundness.

Zhang et al. [ZhangTPDS23] addressed on-chain storage overhead in decentralized storage networks by employing Boneh et al.'s batch polynomial commitment scheme (PCS-MPAP). Their approach compresses multi-file proofs into constant-sized payloads using a vanishing polynomial $Z_T(X) = \prod_{i=1}^m (X - r_i)$. In batch auditing protocols that publish unmasked linear combinations of plaintext polynomial evaluations on-chain (e.g., $E_i = \sum_{l=1}^d \beta_l \cdot S_{l,i}$ and $\{\beta_l\}$ are fixed), a critical privacy vulnerability emerges under the honest-but-curious verifier model. Since the blockchain ledger is fully transparent, an adversary can passively collect audit transcripts across multiple rounds, each utilizing independently sampled folding weights $\{\beta_l^{(t)}\}_{t=1}^T$. By aggregating the corresponding on-chain values $\{E_i^{(t)}\}$, the verifier constructs a linear system $\mathbf{E} = \mathbf{B}\mathbf{S}$. When the number of observed rounds T exceeds the batch size d , the adversary can efficiently recover the underlying evaluation claims $\{S_{l,i}\}$ via Gaussian elimination, thereby reconstructing sensitive polynomial evaluations and violating the statistical confidentiality of the audited data. This cross-round linear correlation attack underscores the necessity of integrating fresh one-time masking or commitment-based hiding into batch proof generation to prevent plaintext leakage on transparent ledgers.

Yu et al. [YuTC25] designed a lightweight storage verification scheme tailored for resource-constrained Industrial IoT environments. Their innovation eliminates bilinear pairings during verification by transmitting quotient polynomial coefficients directly and leveraging public structured parameters. While this achieves computational efficiency for IIoT devices and supports privacy-enhanced verification through masking techniques, the scheme does not support batch auditing across multiple files. Moreover, explicitly publishing quotient coefficients exposes the underlying polynomial structure, enabling curious verifiers to recover original data sectors through Lagrange interpolation with sufficient challenges, thereby compromising statistical hiding properties.

Miao et al. [MiaoSCIS2026] presented a blockchain-assisted multi-keyword searchable PDP scheme optimized for keyword-filtered auditing in cloud environments. Their protocol employs BLS-type homomorphic signatures with scalar linear aggregation, yielding constant-sized proofs independent of batch size. However, treating each sector as an independent scalar rather than encoding chunks as polynomials forfeits algebraic compression benefits. Consequently, both tag generation and verification scale linearly with total sectors, and privacy protection relies on simple random masking without computational hiding guarantees, offering weaker security against adaptive correlation attacks.

Despite these advances, existing schemes face critical limitations: (1) shared evaluation points enable cross-file error cancellation; (2) lack of explicit tight soundness bounds hinders practical parameter selection; (3) insufficient privacy guarantees expose evaluations to curious verifiers; and (4) heavy on-chain computation or complex polynomial arithmetic limits scalability. Our scheme addresses these gaps through Fiat-Shamir folding over independent per-file quotient polynomials, achieving tight soundness bounds, provable evaluation

privacy, and constant-pairing verification without prohibitive computational overhead.

To facilitate a direct comparison, we adopt the unified notation throughout this section: m files in a single audit batch, each comprising n chunks, where each chunk is algebraically encoded as a polynomial of degree $s - 1$ (i.e., s sectors per chunk). Let c denote the number of challenged chunks per file.

1.1) Computation Complexity of Yu et al. [?]

- **Store Algorithm.** For file i and chunk j , the data owner evaluates the chunk polynomial at the secret point and computes the homomorphic tag as ours. Therefore, the storage computation is mnE .
- **ProofGen Algorithm.** The only counted group exponentiation is the privacy mask $B = g^\beta$. Therefore, over m independent files, the computation is mE .
- **Verify Algorithm.** For each file, the verifier reconstructs $H_Z = g^{Z_{\text{prf}}(\psi)}$ from the s -coefficient quotient polynomial, derives H_{prf} , and checks the equation involving B^η , c challenged tags, H_Z^{-z} , and $\text{HHF}(P_{\text{prf}}(z))$. These steps contribute $2s + c + 3$ exponentiations per file. Thus, the verifier computation is $m(2s + c + 3)E$.

1.2) Communication Complexity of Yu et al. [?]

- **Store Algorithm.** The data owner uploads mns field sectors and mn homomorphic tags to the server, and shares one Merkle root per file with the verifier. Hence the one-time storage communication is $mns|p| + mn|G_1| + m|h|$.
- **ProofGen Algorithm.** For each file, the response contains the s -coefficient quotient polynomial, one evaluation value, c challenged tags, one mask B , and c Merkle paths of length $\log n$. Across m files, the proof communication is $m(s + 1)|p| + m(c + 1)|G_1| + mc \log n|h|$.

2.1) Computation Complexity of Zhang et al. [?]

- **Store Algorithm.** For each file block, the tag $\sigma_{i,j} = (H(F_{id}||i||j) \cdot g^{f_{i,j}(\alpha)})^x$ requires one exponentiation for $g^{f_{i,j}(\alpha)}$ and one exponentiation for the outer secret-key power. Across mn tags this gives $2mnE$.
- **ProofGen Algorithm.** For each file, exponentiating the c challenged tags by their audit coefficients costs cE , and applying the file-folding coefficient β_l costs one more exponentiation. Over m files this contributes $mcE + mE$. The two polynomial-commitment proof elements W and W' are obtained from coefficient vectors of length $s - 1$, so their counted work is $(2s - 2)E$. Therefore, the total cost is $(mc + m + 2s - 2)E$.
- **Verify Algorithm.** The verifier constructs the challenge term ζ by exponentiating c hashed indices per file and then applying m file-folding powers, costing $(mc + m)E$. It also forms $\psi = g^{-E_1} \cdot W_1^{-Z_T(z)}$ and $u \cdot v^{-z}$, which contributes $3E$. The batch equation $e(\sigma', g)e(\psi, v) = e(\zeta, v)e(W', u \cdot v^{-z})$ contains four pairings. Hence, verification costs $(mc + m + 3)E + 4P$.

2.2) Communication Complexity of Zhang et al. [?]

- **Store Algorithm.** The storage phase sends the file copy, containing mns field sectors, and mn tags. The storage communication is $mns|p| + mn|G_1|$.
- **ProofGen Algorithm.** The published proof is $(\sigma', W_1, W'_1, E_1)$, namely three group elements and one

field element. Therefore the audit-proof communication is $3|G_1| + |p|$.

3.1) Computation Complexity of Miao et al. [?]

- **Store Algorithm.** In IndexGen, the data owner computes one match index $\Omega_i = (H_{F_i} \cdot H_2(FID_i))^{-x}$ per file, so the counted work is mE . In AuthGen, the data owner creates one authenticator $\sigma_{i,j} = (H_{F_i} \cdot H_2(FID_i) \cdot H_3(au_{i,j}) \cdot u^{m_{i,j}})^x$ for each block. Following the same convention as the scheme's algorithmic cost model, this contributes one counted group exponentiation per authenticator after the hash and multiplication terms are omitted, giving mnE . Therefore the full storage-side computation is $(m + mn)E$, with no pairing. If the keyword-index module is stripped and only the PDP authentication core is compared, this phase reduces to mnE .
- **ProofGen Algorithm.** After locating the matched set R , the server computes $T = \prod_{i \in R} \prod_{j \in Q} \sigma_{i,j}^{e_{i,j}}$, which costs ρcE , and generates the privacy element $R_g = g^r$, which costs one more exponentiation. Thus the proof-generation computation is $(\rho c + 1)E$.
- **Verify Algorithm.** The verifier exponentiates one match index Ω_i for each matched file and c hash terms inside each matched file, and it also computes u^{μ_1} and $R_g^{h_1(T_w, R_g)}$. These operations contribute $(\rho + \rho c + 2)E$. The final verification equation has two pairings, so the verifier computation is $(\rho(c + 1) + 2)E + 2P$.

3.2) Communication Complexity of Miao et al. [?]

- **Store Algorithm.** The data and authenticators sent to the cloud contribute $mn|p| + mn|G_1|$. The match indexes contain one group element per file, adding $m|G_1|$. Thus the normalized storage communication is $mn|p| + (mn + m)|G_1|$, excluding auxiliary searchable index metadata such as addresses and encrypted index matrices.
- **ProofGen Algorithm.** The audit proof (T, μ_1, R_g) consists of two group elements and one field element, so the proof communication is $|p| + 2|G_1|$.

4.1) Computation Complexity of Xu et al. [?]

- **Store Algorithm.** For each file and block, the data owner forms the block polynomial and computes the tag $\sigma_{i,j} = g^{f_{m_{i,j}}(\alpha)}$. Since polynomial evaluation uses field operations, tag generation contributes one group exponentiation for each of the mn tags. The storage-side TagVerify step recomputes each commitment from the s sector coefficients as a product of public-key powers, which costs sE per block tag. Hashing the Merkle leaves and roots is omitted under the current computation metric. Therefore the storage computation is $mn(s + 1)E$, with no pairing.
- **ProofGen Algorithm.** For each file, the CSP linearly combines the c challenged block polynomials, evaluates the aggregate polynomial at the challenge point, and divides by $X - r$. These are field and polynomial operations. The only counted operation is the KZG witness $w_i = g^{h_i(\alpha)}$. Since $h_i(X)$ has degree at most $s - 2$, computing the witness from the structured public key costs $(s - 1)E$ per local proof. Across m local proofs, proof generation costs $m(s - 1)E$, with no pairing.

Table II: Functional and Security Feature Comparison of Batch-Verifiable PDP Schemes

Scheme	Batch Auditing	Independent r_i	Tight Soundness	Evaluation Privacy	Dynamic Updates	Error Resistance
XuTIFS26	✓	×	×	×	✓*	×
ZhangTPDS23	✓	✓	×	×	×	✓
YuTC25	×	-	×	✓†	✓*	×
MiaoSCIS2026	✓	-	×	×	✓	×
Ours	✓	✓	✓	✓	✓	✓

Batch Auditing: Simultaneous multi-file verification.

Tight Soundness: Explicit finite-field error bound, not asymptotic reduction.

Dynamic Updates: Stateless file add/delete with Merkle updates.

-: No independent r_i is required if batch auditing is not supported.

†: Yu et al. [] proofs include quotient polynomial and evaluation point, making the file data exposed to the verifier. The extended version only protects privacy of a data sector (i.e., the constant value of chunk polynomial.)

Independent r_i : Distinct challenge points prevent cross-file pole cancellation.

Evaluation Privacy: Hiding challenged evaluations against curious verifiers.

Error Resistance: Resisting algebraic error-offset and cancellation attacks.

*: Unlike add/delete files, the schemes consider modifying data in a single file

- **Verify Algorithm.** The verifier first aggregates the challenged tags by computing $\prod_{j=1}^c \sigma_{i,a_j}^{v_j}$ for every file, which costs mcE . It then computes $g^{val^{global}}$ and $g^{\alpha-r}$, adding $2E$. The accepted batch path checks one global KZG equation, requiring the two pairings $\hat{e}(\sigma^{global}/g^{val^{global}}, g)$ and $\hat{e}(w^{global}, g^{\alpha-r})$. Thus verification costs $(mc + 2)E + 2P$. The fallback diagnosis used only after a failed global check is not included in this normal verification cost.

4.2) Communication Complexity of Xu et al. []

- **Store Algorithm.** The storage phase sends mn encrypted blocks, each with s field sectors, and mn polynomial tags. The Merkle root for each file is published as the verification anchor, and the algorithm also supplies authentication paths for stored blocks to the corresponding CSP. Therefore the one-time storage communication is $mns|p| + mn|G_1| + mn \log_2 n|h| + m|h|$. If the server reconstructs and stores the Merkle tree locally instead of receiving all paths, the $mn \log_2 n|h|$ term can be omitted.
- **ProofGen Algorithm.** For each file, the proof contains one evaluation value val_i , one witness w_i , the c challenged tags, and c Merkle authentication paths. Across m proofs, the audit-proof communication is $m(|p| + (c + 1)|G_1| + c \log_2 n|h|)$.

VI. EVALUATION

[Overall Acceptance Probability Bound] Let $\rho \in [0, 1]$ denote the fraction of corrupted or missing chunks in any audited file F_i , and let $d = \lfloor \rho n \rfloor$. When the Cloud Server executes ProofGen over a uniformly sampled challenge set $J \subseteq [n]$ with $|J| = c$, the probability that Verify outputs *accept* despite the corruption is bounded by:

$$P_{\text{accept}} \leq \frac{\binom{n-d}{c}}{\binom{n}{c}} + \frac{s}{p} + \text{negl}(\lambda) \approx (1 - \rho)^c + \text{negl}(\lambda). \quad (3)$$

Proof. Let E_{miss} be the event that the challenge set J contains no corrupted chunks. Since J is sampled uniformly without replacement from $[n]$, the probability of this event is exactly the hypergeometric ratio:

$$\Pr[E_{\text{miss}}] = \frac{\binom{n-d}{c}}{\binom{n}{c}}.$$

When $n \gg c$, this simplifies to $(1 - \rho)^c$. In this scenario, the prover can trivially satisfy all verification checks using only intact data.

Let E_{forge} be the complementary event where J intersects at least one corrupted chunk. For Verify to still accept, the adversary must successfully forge Merkle authentication paths, satisfy the quotient-consistency pairing, and produce a valid folded KZG opening. By Theorem 2, under the q -SDH assumption and random oracle model, the probability of bypassing these cryptographic constraints is bounded by $\frac{s}{p} + \text{negl}(\lambda)$.

Since E_{miss} and E_{forge} are mutually exclusive, the total false-acceptance probability satisfies $P_{\text{accept}} = \Pr[E_{\text{miss}}] + \Pr[E_{\text{forge}}]$. Substituting the respective bounds yields the stated inequality. For standard cryptographic parameters ($p \approx 2^{256}$, $s \ll p$), the term $\frac{s}{p}$ is negligible, and the audit security is strictly governed by the sampling parameter c . $\square$

Remark 1 (Parameter Selection). To achieve a target detection rate $1 - \epsilon$ against corruption ratio ρ , the challenge size should satisfy $c \geq \frac{\ln \epsilon}{\ln(1-\rho)}$. For example, guaranteeing 99.9% detection for $\rho \geq 1\%$ requires $c \approx 690$. This bound is independent of the batch size m and applies uniformly across all files in the audit.

VII. CONCLUSION

REFERENCES

- [1] Luo, Z., Jia, Y., Gracia, A.V.O. and Kate, A. Cauchyproofs: Batch-Updateable Vector Commitment with Easy Aggregation and Application to Stateless Blockchains. In *S&P*, 2025, pp. 1947-1963.
- [2] Kate, A., Zaverucha, G.M. and Goldberg, I. Constant-size commitments to polynomials and their applications. In *Asiacrypt*, 2010, pp. 177-194.
- [3] Boneh, D., Drake, J., Fisch, B. & Gabizon, A. Efficient polynomial commitment schemes for multiple points and polynomials. *Cryptology ePrint Archive*. (2020)
- [4] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge," *IACR Cryptology ePrint Archive*, Report 2019/953, 2019. <https://eprint.iacr.org/2019/953>

Table III: Computational Complexity Comparison (Exponentiations E , Multiplications M , and Pairings P)

Scheme	Store/Setup	ProofGen	Verify
Yu et al. [?]	mnE	mE	$m(2s + c + 2)E$
Zhang et al. [?]	$2mnE$	$((mc + m + 2s - 2)E)$	$((mc + m + 3)E + 4P)$
Miao et al. [?]	$((m + mn)E)$	$((\rho c + 1)E)$	$((rho(c + 1) + 2)E + 2P)$
Xu et al. []	$(mn(s + 1)E)$	$(m(s - 1)E)$	$((mc + 2)E + 2P)$
Ours (Proposed)	mnE	$(ms + m + 2s - 1)E$	$(mc + 2m + 3)E + 2P$

Table IV: Communication Complexity Comparison (in bits)

Scheme	Store Phase	ProofGen Phase
Yu et al. [?]	$mns p + mn \mathbb{G}_1 + m h $	$m((s + 1) p + (c + 1) \mathbb{G}_1 + c \log_2 n h)$
Zhang et al. [?]	$mns p + mn \mathbb{G}_1 $	$3 \mathbb{G}_1 + p $
Miao et al. [?]	$mn p + (mn + m) G_1 $	$ p + 2 G_1 $
Xu et al. [?]	$mns p + mn G_1 + mn \log_2 n h + m h $	$m(p + (c + 1) G_1 + c \log_2 n h)$
Ours (Proposed)	$mns \mathbf{p} + mn \mathbb{G}_1 + \mathbf{m} \mathbf{h}$	$\mathbf{m}((\mathbf{c} + 2) \mathbb{G}_1 + \mathbf{p} + \mathbf{c} \log_2 \mathbf{n} \mathbf{h}) + 4 \mathbb{G}_1 + 2 \mathbf{p}$